

value should *not* have spaces around it, to show that these are not typical assignment statements. These lines are properly formatted:

```
def orderBurgers(nBurgers, ketchup=True, mustard=True, pickles=True):  
  
    orderBurgers(2, mustard=False)
```

These lines will also work fine, but they don't follow the formatting convention and are less readable:

```
def orderBurgers(nBurgers, ketchup = True, mustard = True, pickles = True):  
  
    orderBurgers(2, mustard = False)
```

When calling a function that has both positional parameters and keyword parameters, you must provide values for all the positional parameters first, before any optional keyword parameters.

Keyword arguments in calls can be specified in any order. Calls to our `orderBurgers()` function could be made in various ways, such as:

```
orderBurgers(2, mustard=False, pickles=False) # only ketchup
```

or:

```
orderBurgers(2, pickles=False, mustard=False, ketchup=False) # plain
```

All keyword parameters will be given the appropriate values, independent of the order of the arguments.

While all the default values in the `orderBurgers()` example were Boolean values, a keyword parameter can have a default value of any data type. For example, we could write a function to allow a customer to make an ice cream order like this:

```
def orderIceCream(flavor, nScoops=1, coneOrCup='cone', sprinkles=False):
```

The caller must specify a flavor, but by default will get one scoop in a cone with no sprinkles. The caller could override these defaults with different keyword values.